Abstract

This report provides details about how patrons move around the theme park and get into queues for each of the rides.

Timestep 10:

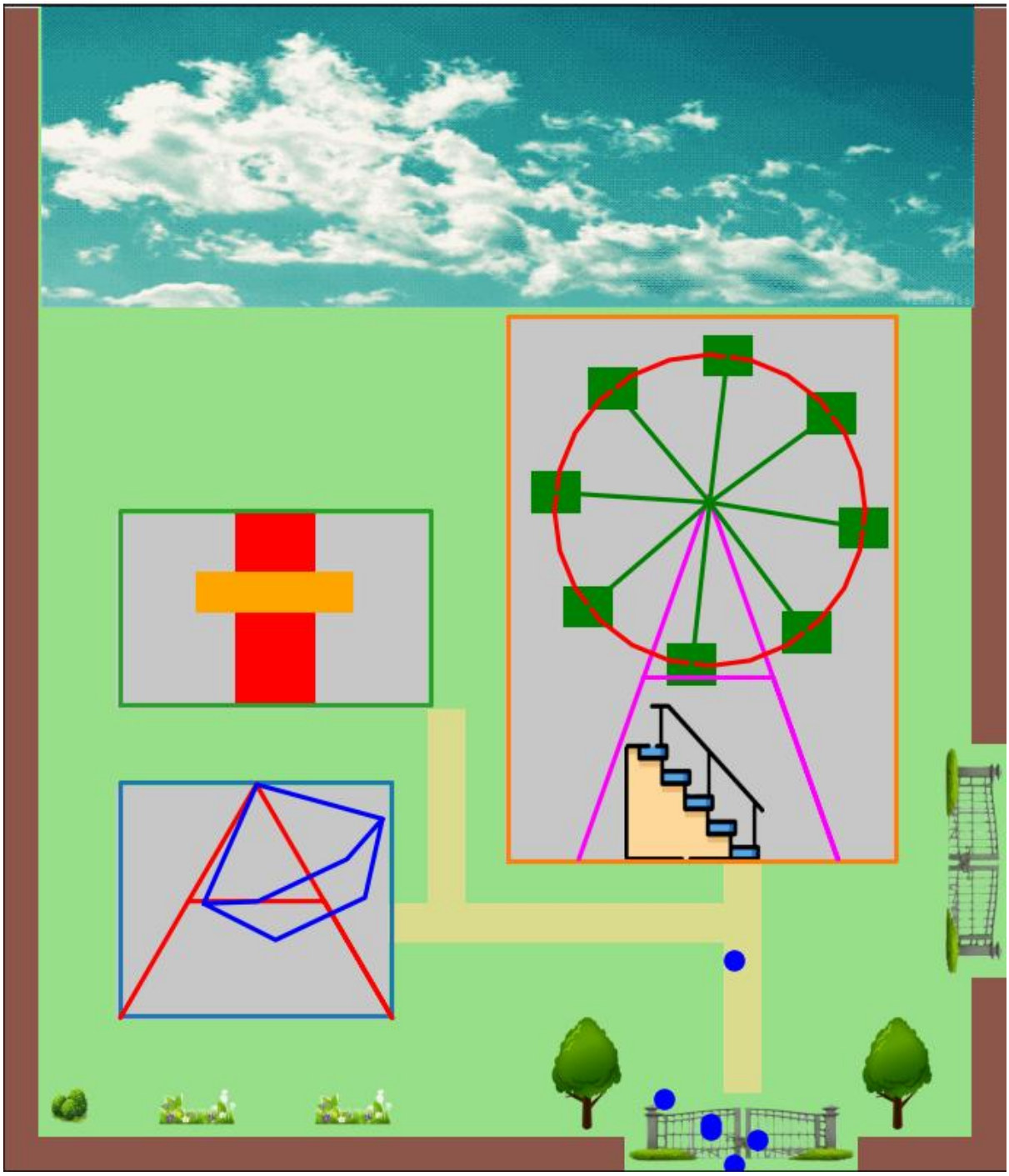


Figure 1

Table of Contents

Abstract 0
Overview 3
User Guide 4
Traceability Matrix 5
Discussion 26
UML Diagram 27
..... 28
..... 28
..... 28
..... 28
..... 28
Module Description 28
Showcase 30
Scenario 1 30
Scenario 2 32
Scenario 3 34
Conclusion 35
Future Work 36
References 36

Table of Figures

Figure 1 0
Figure 2 3
Figure 3 7
Figure 4 8
Figure 5 9
Figure 6 9
Figure 7 9
Figure 8 9
Figure 9 10
Figure 10 10
Figure 11 11
Figure 12 12
Figure 13 12
Figure 14 14

Figure 15 14
Figure 16 15
Figure 17 15
Figure 18 16
Figure 19 17
Figure 20 19
Figure 21 19
Figure 22 20
Figure 23 20
Figure 24 21
Figure 25 21
Figure 26 24
Figure 27 24
Figure 28 24
Figure 29 25
Figure 30 25
Figure 31 25
Figure 32 25
Figure 33 26
Figure 34 26
Figure 35 26
Figure 36 31
Figure 37 31
Figure 38 32
Figure 39 32
Figure 40 33
Figure 41 33
Figure 42 34
Figure 43 34
Figure 44 35
Figure 45 35

Overview

The assignment is about the simulation of a theme park called 'Adventure World' which comprises of three rides with different movements. The theme park has three main attractions namely Pirate Ship, Ferris Wheel and Drop Tower. There are some decorative images used to enhance the appearance of the park; two trees by the entrance, a bush, two flower fields and two gates, one representing the entrance and the other representing the exit. The simulation aims to display the rides and the patrons which spawn and navigate across the terrain, with barriers defining the park limits. The main program adventureworld.py runs with the help of Backcode.py which contains all the classes needed.

Time step 10

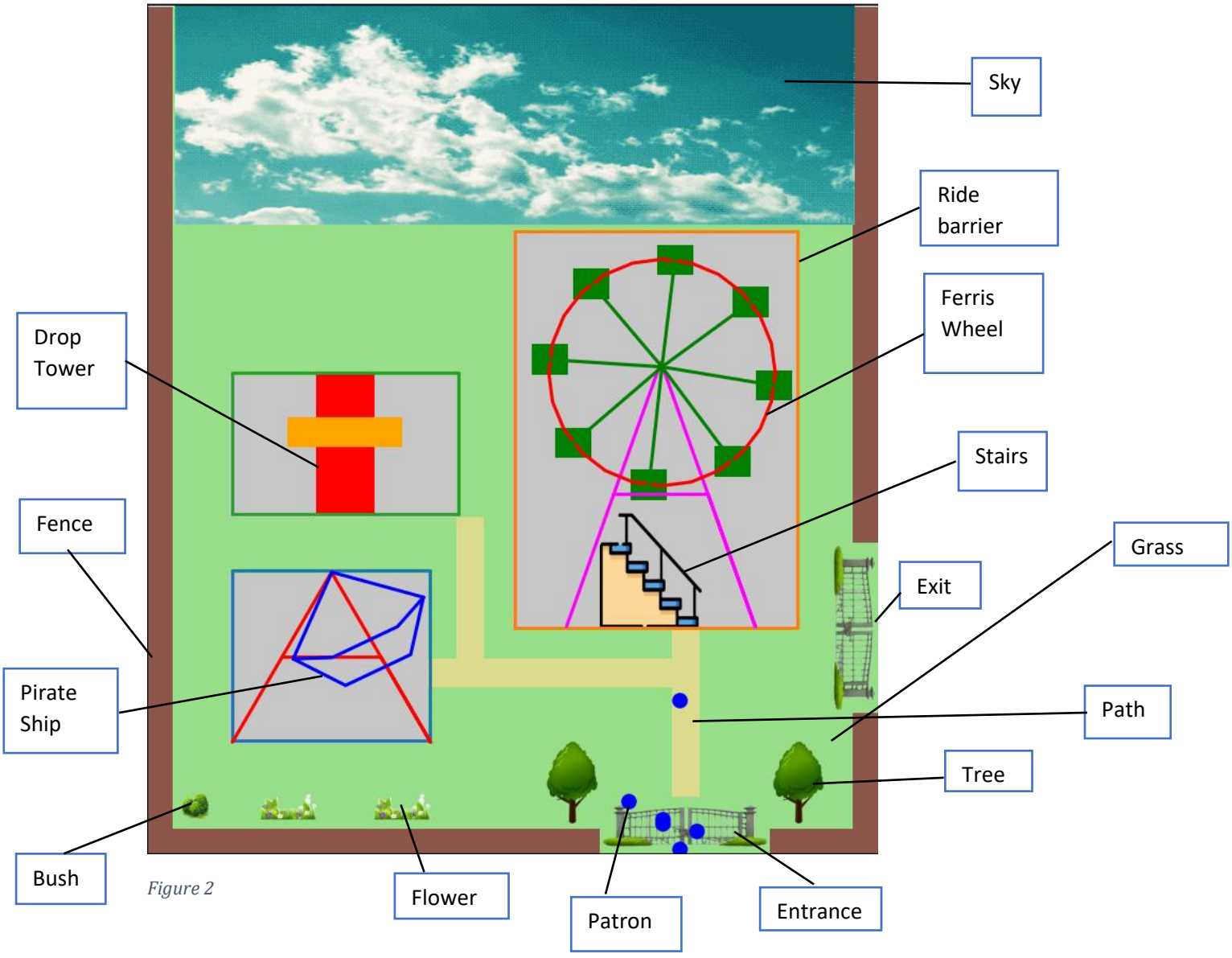


Figure 2

User Guide

Instructions on how to use your simulation – setup, required packages, code usage (and parameter sweep code usage, if applicable)

In order to ensure the smooth running of the program, the files below should be inside the same directory:

- AdventureWorld.py
- bush.jpg
- exit.jpg
- flowers_2.jpg
- gate.jpg
- partly_cloudy_sky_GIF.gif
- stairs.png
- ThemePark.py
- tree.png

How to run the program:

Python version: 3.11.9

The required Python libraries which need to be installed are listed below:

- numpy
- matplotlib
- random
- ListedColormap
- Pillow

If you are running the program on an integrated development environment (IDE):

Run the file entitled: AdventureWorld.py

If you are running the program on the terminal:

>>>>>: python3 AdventureWorld.py

On the outset of the program, you will be prompted to enter the number of patrons, then for each patron the user will be given a choice of whether to roam or to ride. After choosing which attraction patrons will ride, then the simulation starts. The patrons are either shown moving to queue near the barriers of the concerned rides or roaming without entering within the boundaries of the rides.

Traceability Matrix

Here are tables giving overview of features, and the implementation and testing of the code. Snapshots of the code follow the tables.

Feature	Code Reference	Test Reference	Status	Date Completed
1.0 Ride				
1.1 Ride	Class Ride in ThemePark.py	This class was created to test the plotting and movement of the ride objects	p	18/09/25
1.1.1 Initialise	def __init__(self, name, size, position, duration, park, capacity=8, animation_count=0) Refer to figure 3	This method was created to initialise the parameters	P	18/09/25
1.1.2 Move patrons from ship queue to ship	def update_riders_ship(self, park) Refer to figure 4	N/A	S	N/A
1.1.3 Move patrons from ship queue to ship	def update_riders_ferris(self, park) Refer to figure 5	N/A	S	N/A
1.1.4 Start ride if riders are loaded	start_ship(self) Refer to figure 6	N/A	S	N/A
1.1.5 Start ride if riders are loaded	def start_ferris(self) Refer to figure 7	N/A	S	N/A
1.1.6 Update ship state each timestep	def update_ship(self, park) Refer to figure 8	This method was only able to load riders in the list for riders from the queue but could not display them on onboard	S	18/09/25

1.1.7 Update ferris state each timestep	def update_ferris(self, park) Refer to figure 9	This method was only able to load riders in the list for riders from the queue but could not display them on onboard	S	18/09/25
1.1.8 Plot boundary Plot and swing pirate ship	def plot_ship(self, plt, pos) Refer to figure 10 (COMP1005, 2025)	This method was able to plot the boundary of the ship, plot and update the position of the ship. It was tested by swinging the ship within the boundary.	P	18/09/25
1.1.9 Plot boundary Plot and move drop tower	def drop_tower(self, p) Refer to figure 11 (COMP1005, 2025)	This method was able to plot the boundary of the drop tower, plot and update the position of the seat part of the drop tower. It was tested by moving the seat part of drop tower within a y coordinate limit (75-80).	P	18/09/25
1.1.10 Plot boundary Plot and rotate ferris wheel	def plot_ferrisWheel(self, p, pos) Refer to figure 12 (COMP1005, 2025)	This method was able to plot the boundary of the ferris wheel, plot and update the position of the ferris wheel. It was tested by rotating the lines of the ferris wheel within the boundary.	P	18/09/25

```python
def __init__(self, name, size, position, duration, park, capacity=8, animation_count=0):
    """
    Parameters:
    name: string, name of the ride
    size: tuple (width, height)
    position: tuple (x, y), location of ride
    duration: int, duration of the ride in timesteps
    capacity: int, maximum patrons allowed on 1 ride
    animation_count: int, counter for animation (starts at 0)
    """
    self.name = name
    self.size = size
    self.position = position
    self.duration = duration
    self.capacity = capacity
    self.animation_count = animation_count # increases only when running
    self.park = park

    # drop tower properties
    self.tower_x = 30
    self.tower_y = 60
    self.width_tower = 10
    self.height_tower = 25
    self.tower_colour = 'red'

    self.width_seat = 20
    self.height_seat = 5
    self.seat_x = 25
    self.seat_y = 70
    self.seat_colour = 'orange'
    self.direction = 1
    width_seat = 20
    height_seat = 5
    # Ride state management
    self.state = "idle" # either idle, running, loading
    self.ship_riders = [] # patrons on ship
    self.ferris_riders = [] # patrons on ferris wheel
    self.count = 0 # increases only for current state
    self.loading_time = 20 # timesteps taken to load patrons
    self.unloading_time = 15 # timesteps taken to unload

    self.waypoints = {
        'entrance': (94, 5),
        'ferris_wheel': (100, 40),
        'ferris_junction': (94, 30),
        'pirate_ship': (51, 30),
        'pirate_junction': (56, 35),
        'drop_junction': (56, 60),
        'exit': (124, 25)
    }
```

Figure 3

```python
def __init__(self, name, size, position, duration, park, capacity=8, animation_count=0):
    """
    Parameters:
    name: string, name of the ride
    size: tuple (width, height)
    position: tuple (x, y), location of ride
    duration: int, duration of the ride in timesteps
    capacity: int, maximum patrons allowed on 1 ride
    animation_count: int, counter for animation (starts at 0)
    """
    self.name = name
    self.size = size
    self.position = position
    self.duration = duration
    self.capacity = capacity
    self.animation_count = animation_count # increases only when running
    self.park = park

    # drop tower properties
    self.tower_x = 30
    self.tower_y = 60
    self.width_tower = 10
    self.height_tower = 25
    self.tower_colour = 'red'

    self.width_seat = 20
    self.height_seat = 5
    self.seat_x = 25
    self.seat_y = 70
    self.seat_colour = 'orange'
    self.direction = 1
    width_seat = 20
    height_seat = 5
    # Ride state management
    self.state = "idle" # either idle, running, loading
    self.ship_riders = [] # patrons on ship
    self.ferris_riders = [] # patrons on ferris wheel
    self.count = 0 # increases only for current state
    self.loading_time = 20 # timesteps taken to load patrons
    self.unloading_time = 15 # timesteps taken to unload

    self.waypoints = {
        'entrance': (94, 5),
        'ferris_wheel': (100, 40),
        'ferris_junction': (94, 30),
        'pirate_ship': (51, 30),
        'pirate_junction': (56, 35),
        'drop_junction': (56, 60)
    }
```

Figure 4

```python
def update_riders_ship(self, park):
    """
    Move patrons from ship queue to ship
    park: Park_Operations which has queue_ship
    """
    if self.state == "idle":
        while len(self.park.queue_ship) > 0 and len(self.ship_riders) < self.capacity:
            patron = self.park.queue_ship.pop(0) # Take 1st patron from queue
            self.ship_riders.append(patron) # put 1st person on ride
            self.park.queue_ship.remove(patron)
            patron.state = "loading"
            print(f"{patron[0]} boarded {self.name}")
            for i, p in enumerate(self.park.queue_ship): # Update queue_position for the rest of patrons
                p.queue_position = i
```

Figure 5

```python
def update_riders_ferris(self, park):
    """
    Move patrons from ship queue to ship
    """
    if self.state == "idle":
        while len(self.park.queue_ferris) > 0 and len(self.ferris_riders) < self.capacity:
            patron = self.park.queue_ferris.pop(0) # Take 1st patron from queue
            self.ferris_riders.append(patron) # put 1st person on ride
            patron.state = "loading"
            print(f"{patron[0]} boarded {self.name}")
            for i, p in enumerate(self.park.queue_ferris): # Update queue_position for the rest of patrons
                p.queue_position = i
```

Figure 6

```python
def start_ship(self):
    """
    Start ride if riders are loaded
    """
    if len(self.ship_riders) > 0:
        self.state = "running"
        self.time_remaining = self.duration
        print(f"{self.name} started with {len(self.ship_riders)} riders.")
```

Figure 7

```python
def start_ferris(self):
    """
    Start ride if riders are loaded
    """
    if len(self.ferris_riders) > 0:
        self.state = "running"
        self.time_remaining = self.duration
        print(f"{self.name} started with {len(self.ferris_riders)} riders.")
```

Figure 8

```
def update_ship(self, park):
    """
    Update ship state each timestep
    """
    if self.state == "running":
        self.time_remaining -= 1
        self.animation_count += 1 # increase for animation
        if self.time_remaining <= 0:
            #self.finish_ship() # 1 round of ride finished
            self.state = "unloading"
    elif self.state == "idle":
        if len(park.queue_ship) > 0: # verify if there is still a queue
            self.state = "loading"
            self.update_riders_ship(park)
            #self.start_ship()
```

Figure 9

```
def update_ferris(self, park):
    """
    Update ferris state each timestep
    """
    if self.state == "running":
        self.time_remaining -= 1
        self.animation_count += 1 # increase for animation
        if self.time_remaining <= 0:
            #self.finish_ferris() # 1 round of ride finished
            self.state = "unloading"
    elif self.state == "idle":
        if len(park.queue_ferris) > 0: # verify if there is still a queue
            self.state = "loading"
            self.update_riders_ferris(park)
            self.start_ferris()
```

Figure 10

```
def plot_ship(self, plt, pos):
    """
    Plot boundary
    Plot and swing pirate ship
    """
    # plot ship inside of box
    xShip, yShip = self.position
    widthShip, heightShip = self.size
    x = np.array([xShip, xShip + widthShip, xShip + widthShip, xShip, xShip])
    y = np.array([yShip, yShip, yShip + heightShip, yShip + heightShip, yShip])
    plt.plot(x,y) # Plot Ship box
    frame_x = np.array([1, 2, 3, 4, 5, 4, 2])
    frame_y = np.array([1, 3, 5, 3, 1, 3, 3])
    #p.plot(frame_x,frame_y) # frame

    ship_x = np.array([1, 2, 4, 5, 3, 1, 2, 4, 5])
    ship_y = np.array([3, 2.5, 2.5, 3, 5, 3, 1.5, 1.5, 3])

    x_ship_scaled = (ship_x/5.5) * widthShip
    y_ship_scaled = (ship_y/5.5) * heightShip


    x_frame_scaled = (frame_x/4) * widthShip
    y_frame_scaled = (frame_y/4) * heightShip


    x_frame_translated = x_frame_scaled + ((1.7/4) * xShip)
    y_frame_translated = y_frame_scaled + ((2.45/4) * yShip)
    plt.plot(x_frame_translated, y_frame_translated,solid_capstyle="round", color='red')

    x_ship_translated = x_ship_scaled + ((3.6/4) * xShip)
    y_ship_translated = y_ship_scaled + ((4.5/4) * yShip)

    theta = np.radians(pos)


    # (32.7, 50) is like the nail fixing the ship to the frame
    x_move = (x_ship_translated - 32.7)*np.cos(theta) - (y_ship_translated - 50)*np.sin(theta) + 32.7
    y_move = (x_ship_translated - 32.7)*np.sin(theta) + (y_ship_translated - 50)*np.cos(theta) + 50
    plt.plot(x_move, y_move,solid_capstyle="round", color='blue')
```

Figure 11

```python
def drop_tower(self, p):
    """
    Plot boundary
    Plot and move drop tower
    """
    xShip, yShip = self.position
    widthShip, heightShip = self.size
    x = np.array([xShip, xShip + widthShip, xShip + widthShip, xShip, xShip])
    y = np.array([yShip, yShip, yShip + heightShip, yShip + heightShip, yShip])
    p.plot(x,y) # Plot drop tower box


    tower = patches.Rectangle((self.tower_x, self.tower_y), self.width_tower, self.height_tower, color=self.tower_colour)
    p.gca().add_patch(tower)

    seat = patches.Rectangle((self.seat_x, self.seat_y), self.width_seat, self.height_seat, color=self.seat_colour)
    p.gca().add_patch(seat)

    if self.seat_y >= 80:
        self.direction = -1
    elif self.seat_y <= 70:
        self.direction = 1
    self.seat_y += 2 * self.direction # speed
```

Figure 12

```python
def plot_ferrisWheel(self, p, pos):
    """
    Plot boundary
    Plot and rotate ferris wheel
    """
    xShip, yShip = self.position
    widthShip, heightShip = self.size
    x = np.array([xShip, xShip + widthShip, xShip + widthShip, xShip, xShip])
    y = np.array([yShip, yShip, yShip + heightShip, yShip + heightShip, yShip])
    p.plot(x,y) # Plot Ferris Wheel box

    frame_x = np.array([1, 2, 3, 4, 5, 4, 2])
    frame_y = np.array([1, 3, 5, 3, 1, 3, 3])

                        (variable) frame_x: ndarray[tuple[Any, ...], dtype[Any]]
    x_frame_scaled = (frame_x/6) * widthShip
    y_frame_scaled = (frame_y/6) * heightShip

    x_frame_translated = x_frame_scaled + ((8.1/8) * xShip)
    y_frame_translated = y_frame_scaled + ((5.7/8) * yShip)
    p.plot(x_frame_translated,y_frame_translated,solid_capstyle="round", color='magenta')

    pie = np.pi
    angle_rad = np.linspace(0, 2 * pie, 25) # 0 to 2 pi rad
    r = 20
    x_wheel = r * np.cos(angle_rad) # x pts
    y_wheel = r * np.sin(angle_rad) # y pts
    x_wheel_translated = x_wheel + ((9.2/8) * xShip) + (1/4 * xShip)
    y_wheel_translated = y_wheel + ((17/8) * yShip)

    p.plot(x_wheel_translated,  y_wheel_translated, color='r')

    range_angles = np.linspace(0, 2 * pie, 8, endpoint=False) # angles in rad from 0 to 360
    for i in range_angles:
        rad = pos + i
        x_line = (r * np.cos(rad)) + ((9.2/8) * xShip) + (1/4 * xShip)# x pts for line with translation
        y_line = (r * np.sin(rad)) + ((17/8) * yShip) # y pts for line with translation

        p.plot([91,  x_line], [86, y_line], color='green') # plot from centre to edge of wheel
        cabin_width = 6
        cabin_height= 5
        cabin =  plt.Rectangle((x_line - (cabin_width/2), y_line - (cabin_height/2)), cabin_width, cabin_height, color='g', linewidth=1)

        p.gca().set_aspect('equal', adjustable = 'box')
        p.gca().add_patch(cabin)
```

Figure 13

1.2.0 Terrain	Class Terrain in ThemePark.py	This class was created to generate the terrain(background) of the theme park	P	19/09/25
1.2.1 Initialise	def __init__ (self, boundary, size, entrance, exit, gif_file=None) Refer to figure 13	This method was used to initialise the parameters of the terrain.	P	19/09/25
1.2.2 Load all frames from gif	def load_gif_frames(self, filename) Refer to figure 14	This method was used to handle the gif file and load all its frames	P	19/09/25
1.2.3 Shows which GIF frame should be displayed	def update_frame(self, timestep) Refer to figure 15	This method was used to display the frame when the timestep is increasing	P	19/09/25
1.2.4 Generates terrain grid with numbers representing different areas	def generate_grid(self) Refer to figure 16	This method was used to generate the terrain. It was tested by displaying grass, the paths, fence and ride area.	P	19/09/25
1.2.5 Verifies if position (x, y) is walkable ()	def is_walkable(self, x, y) Refer to figure 17	This method was used to check whether a patron can walk, that is it can walk on grass and paths but not fence and ride area when roaming.	P	19/09/25
1.2.6 Stores pictures and display them as decorations on terrain	def plot_terrain(self, p) Refer to figure 18	This method was used to show small pictures as decorations. It was tested by being display on the coordinates given.	P	19/09/25

```python
def __init__ (self, boundary, size, entrance, exit, gif_file=None):
    """
    boundary: tuple, fences of the theme park (x, y)
    size: tuple, width and length of the theme park (x, y)
    entrance: tuple, entrance (x, y)
    exit: tuple, exit (x, y)
    gif_file: string, gif path
    """
    self.boundary = boundary
    self.size = size
    self.entrance = entrance
    self.exit = exit
    self.terrain_grid = self.generate_grid()

    self.background_gif = gif_file # gif file path
    self.gif_frames = [] # store all frames
    self.current_frame_index = 0 # shows current frame
    self.frame_delay = 3 # shows frame each 3 timestep

    if gif_file: # load gif
        self.load_gif_frames(gif_file)
```

Figure 14

```python
def load_gif_frames(self, filename):
    """
    Load all frames from gif
    """
    try:
        gif = Image.open(filename)
        if hasattr(gif, 'n_frames'): # gets all frames of gif
            total_frames = gif.n_frames
        else:
            total_frames = 1

        for frame_num in range(total_frames):
            gif.seek(frame_num) # go to this frame
            frame = gif.convert('RGBA')
            frame_array = np.array(frame)
            self.gif_frames.append(frame_array) # store frame

        print(f"Loaded {len(self.gif_frames)} frames")
    except FileNotFoundError:
        print(f"Error: GIF file '{filename}' not found.")
        self.gif_frames = []
    except Exception as e:
        print(f"Error loading GIF: {e}")
        self.gif_frames = []
```

Figure 15

```python
def update_frame(self, timestep):
    """
    Shows which GIF frame should be displayed
    """
    if len(self.gif_frames) > 0:
        total_frames = len(self.gif_frames)
        self.current_frame_index = (timestep // self.frame_delay) % total_frames
```

Figure 16

```python
def generate_grid(self):
    """
    Generate terrain grid with numbers representing different areas
    Grid shape: (130, 120) = (x_max, y_max)
    Indexing: grid[x, y] matches matplotlib coordinates

    Grid codes
    0 = grass (walkable)
    1 = fence (not walkable)
    2 = Ride area (not walkable)
    3 = path (walkable)
    """
    grid_x = 130
    grid_y = 150
    grid = np.zeros((grid_x, grid_y)) # Shape matches (x_limit, y_limit)
    grid[:,:] = 1 # Everything brown, the fence
    grid[5:125,5:150] = 0 # grass
    grid[15:50, 20:50] = 2 # Background of pirate ship
    grid[65:115, 40:110] = 2 # Background of ferris wheel
    grid[15:55, 60:85] = 2 # background of drop down
    grid[93:98, 0:40] = 3 # path to ferris wheel
    grid[50:93, 30:35] = 3 # path to pirate ship
    grid[55:60, 35:60] = 3 # path to drop tower
    grid[111:119, 6:19] = 1 # tree
    grid[71:79, 6:19] = 1 # tree
    grid[6:124, 112:149] = 1   # sky

    return grid
```

Figure 17

```python
def is_walkable(self, x, y):
    """
    Verifies if position (x, y) is walkable ()
    Uses matplotlib coordinates directly - grid[x, y]

    Parameters:
    x: x-coordinate (0-130)
    y: y-coordinate (0-120)

    Returns True if position is movable (grass or path)
    """
    # Convert to integers for array indexing
    x_int = int(x)
    y_int = int(y)

    # Check bounds
    if 0 <= x_int < 130 and 0 <= y_int < 120:
        # fence[x, y] matches matplotlib (x, y)
        cell_value = self.terrain_grid[x_int, y_int] # Check if (x_int & y_int) match 0,1,2,3
        return cell_value in [0, 3] # In bounds

    return False # Out bounds (not walkable)
```

Figure 18

```python
def plot_terrain(self, p):
    """
    Stores pictures and gif, and display them on terrain
    """

    xpos, ypos= self.boundary
    width, height = self.size
    x = np.array([xpos, xpos + width, xpos + width, xpos, xpos])
    y = np.array([ypos, ypos, ypos + height, ypos + height, ypos])
    #p.plot(x,y) # Plot boundary

    # plot grid after transposing (width,height become height,width)
    p.imshow(self.terrain_grid.T, origin="lower", cmap=cmap)

    x_en, y_en = self.entrance
    #ax.add_patch(plt.Rectangle((x_en , y_en), 20, 5, color=tab20[5])) # entrance gate
    entrance_gate = mpimg.imread("gate.jpg")
    p.imshow(entrance_gate, extent=[x_en, x_en + 30, y_en, y_en + 10])

    exit = mpimg.imread("exit.jpg")
    p.imshow(exit, extent=[120, 130, 25, 55])

    tree = mpimg.imread("tree.png")
    p.imshow(tree, extent=[110, 120, 5, 20])
    p.imshow(tree, extent=[70, 80, 5, 20])


    flower = mpimg.imread("flowers_2.jpg")
    p.imshow(flower, extent=[20, 30, 6, 11])
    p.imshow(flower, extent=[40, 50, 6, 11])

    bush = mpimg.imread("bush.jpg")
    p.imshow(bush, extent=[6, 11, 6, 11])

    stairs = mpimg.imread("stairs.png")
    p.imshow(stairs, extent=[80, 98, 40, 62])

    if len(self.gif_frames) > 0:
        current_frame = self.gif_frames[self.current_frame_index]
        p.imshow(current_frame, extent=[5, 125, 111, 150], alpha=1, aspect='auto')
```

Figure 19

1.3.0 Patron	Class Patron in ThemePark.py	This class was created to plot and move patrons.	P	20/09/25
Initialise	def __init__ (self, name, marker, position, park) Refer to figure 19	This method was created to initialise the parameters of Patrons	P	20/09/25
1.3.1 Plot patron at entrance	def plot_patron(self, p) Refer to figure 20	N/A	S	N/A
1.3.2 Update patron position	def step_change(self, terrain) Refer to figure 21	This method was used to move a patron depending on its state	P	20/09/25

		(roaming, riding, leaving). It was tested by checking whether they are walking on walkable grounds and if they are arriving at their intended destination.		
1.3.3 Move patron randomly (roaming) while stay on walkable terrain	def random_walk(self, terrain) Refer to figure 22	This method was used to calculate random coordinates for the random movement of patrons. It was tested by displaying walking randomly while on walkable grounds.	P	20/09/25
1.3.4 Move 1 step toward target with is.walkable(self, x, y)	def move_toward_target(self, target, terrain) Refer to figure 23	This method was used to calculate coordinate to make patrons move in a linear fashion to their destination. It was tested by making the patrons move in queues to their chosen rides.	P	20/09/25
1.3.5 Move 1 step toward target without is.walkable(self, x, y)	def move_toward_ride(self, target) Refer to figure 24	This method was used to calculate coordinate to make patrons move in a linear fashion to their destination without considering walkable grounds. It was tested by making the patrons move in queues to their chosen rides.	P	20/09/25

```python
def __init__ (self, name, marker, position, park):
    """
    name: string, name of patron
    marker: shape used to represent the patron
    """
    self.patron = []
    self.name = name
    self.position = position
    self.marker = marker
    self.park = park
    # state management
    self.state = None
    self.target_waypoint = None # coordinates of chosen ride
    self.chosen_ride = None # ride chosen by patron
    self.queue_position = None # position in queue

    self.x = random.uniform(94, 94) # spawn patrons at entrance
    self.y = random.uniform(0, 10)
    #self.x, self.y = position

    # waypoints directory
    self.waypoints = {
        'entrance': (94, 5),
        'ferris_wheel': (100, 40),
        'ferris_junction': (94, 30),
        'pirate_ship': (51, 30),
        'pirate_junction': (56, 35),
        'drop_junction': (56, 60),
        'exit': (124, 25)
    }
```

Figure 20

```python
def plot_patron(self, p):
    """Plot patron at entrance"""

    p.plot(self.x, self.y, marker=self.marker, color='blue')
```

Figure 21

```python
def step_change(self, terrain):
    """
    Update patron position
    Verifies terrain for walkability using direct coordinate access
    """
    if self.state == "roaming":
        # Random walk respecting terrain
        self.random_walk(terrain)
        distance = ((self.x - self.waypoints['exit'][0])**2 + (self.y - self.waypoints['exit'][1])**2)**0.5
        if distance < 5:
            self.park.despawn_patron(self)

    elif self.state == "riding_ship":
        # move to pirate ship
        self.move_toward_ride(self.waypoints['pirate_ship'])
        distance = ((self.x - self.waypoints['pirate_ship'][0])**2 + (self.y - self.waypoints['pirate_ship'][1])**2)**0.5
        if distance < 5:
            # Get in line if patron within 5 units
            self.park.make_queue_ship(self)
    elif self.state == "riding_ferris":
        # move to pirate ship
        self.move_toward_ride(self.waypoints['ferris_wheel'])
        distance = ((self.x - self.waypoints['ferris_wheel'][0])**2 + (self.y - self.waypoints['ferris_wheel'][1])**2)**0.5
        if distance < 5:
            # Get in line if patron within 5 units
            self.park.make_queue_ferris(self)
    elif self.state == "riding_tower":
        # move to pirate ship
        self.move_toward_ride(self.waypoints['drop_junction'])
        distance = ((self.x - self.waypoints['drop_junction'][0])**2 + (self.y - self.waypoints['drop_junction'][1])**2)**0.5
        if distance < 5:
            # Get in line if patron within 5 units
            self.park.make_queue_drop(self)

    elif self.state == "leaving":
        # Walk toward exit
        self.move_toward_ride(self.waypoints['exit'])
        distance = ((self.x - self.waypoints['exit'][0])**2 + (self.y - self.waypoints['exit'][1])**2)**0.5
        if distance < 5:
            self.park.despawn_patron(self)
```

Figure 22

```python
def random_walk(self, terrain):
    """
    Move patron randomly (roaming) while stay on walkable terrain
    Uses terrain.is_walkable()
    """
    validmoves = [(3,0),(3,3),(-3,-3),(3,-3),(0,3),(0,0),(0,-3),
                  (-3,0),(-3,3)]     # list of valid moves
    move = random.choice(validmoves)       # randomly choose a move
    print((self.x, self.y), move)
    # need to update the position based on the move
    new_x = self.x + move[0]
    new_y = self.y + move[1]

    # Verify if new position is walkable (using matplotlib coordinates)
    if terrain.is_walkable(new_x, new_y):
        self.x = new_x
        self.y = new_y
    return (new_x, new_y)
```

Figure 23

```python
def move_toward_target(self, target, terrain):
    """
    Move 1 step toward target
    """
    target_x, target_y = target

    # Calculate direction
    dx = target_x - self.x
    dy = target_y - self.y
    distance = (dx**2 + dy**2)**(0.5) # pythagoras theorem

    if distance < 1:
        # Reached target
        return True # stops walking

    step_size = 1
    # take 1 step in direction of target
    new_x = self.x + ((dx / distance) * step_size)
    new_y = self.y + ((dy / distance) * step_size)

    # Verify if new position is walkable (using matplotlib coordinates)
    if terrain.is_walkable(new_x, new_y):
        # patron moves each time it is on path or grass
        self.x = new_x
        self.y = new_y

    return False # Has not reached target but moved 1 step closer
```

Figure 24

```python
def move_toward_ride(self, target):
    """
    Move 1 step toward target
    """
    target_x, target_y = target

    # Calculate direction
    dx = target_x - self.x
    dy = target_y - self.y
    distance = (dx**2 + dy**2)**(0.5) # pythagoras theorem

    if distance < 1:
        # Reached target
        return True # stops walking

    step_size = 1
    # take 1 step in direction of target
    new_x = self.x + ((dx / distance) * step_size)
    new_y = self.y + ((dy / distance) * step_size)

    self.x = new_x
    self.y = new_y
```

Figure 25

1.4.0 Park_Operations	Class Park_Operations In ThemePark.py	This class was created to handle operations concerning patrons and the rides.	P	21/09/25
1.4.1 Initialise	def __init_(self, terrain) Refer to figure 25	This class was created to initialise the parameters of Park_Operations	P	21/09/25
1.4.2 Spawn patron	spawn_patron(self) Refer to figure 26	The method was used to spawn a patron and append it to a list. It was tested by displaying the patron at the entrance.	P	21/09/25
1.4.3 Make queue for pirate ship (add patron to queue)	def make_queue_ship(self, patron) Refer to figure 27	The method was used to add patrons to the pirate ship queue and assign them their respective position. It is tested by visually plotting in a horizontal line outside the pirate ship barrier.	P	21/09/25
1.4.4 Make queue for ferris wheel (add patron to queue)	def make_queue_ferris(self, patron) Refer to figure 28	The method was used to add patrons to the ferris wheel queue and assign them their respective position. It is tested by visually plotting in a horizontal line	P	21/09/25

		outside the ferris wheel barrier.		
1.4.5 Make queue for drop wheel (add patron to queue)	def make_queue_drop(self, patron) Refer to figure 29	The method was used to add patrons to the drop tower queue and assign them their respective position. It is tested by visually plotting in a horizontal line outside the drop tower barrier.	P	21/09/25
1.4.6 Update movement of all patrons in list for one timestep	def move_all_patrons(self) Refer to figure 30	The method was used to move all patrons in list for each timestep according to their state.	P	21/09/25
1.4.7 Update both rides	def update_all_rides(self) Refer to figure 31	The method was used to store update_ship(self) and update_ferris(self) to use both above at the same time	S	21/09/25
1.4.8 Despawn patron	def despawn_patron(self, patron) Refer to figure 32	N/A	S	N/A
1.4.9 Plot all active patrons	def plot_all_patrons(self, p) Refer to figure 33	The method was used to display all active patrons. It was tested by display the patrons whether they are roaming, queueing or leaving.	P	21/09/25

```python
def __init__(self, terrain):
    self.terrain = terrain
    self.patrons_list = [] # List of all active patrons
    self.rides = [] # list of all rides
    self.queue_ship = []
    self.queue_ferris = []
    self.queue_drop = []
    self.queue_position = None
    self.waypoints = {
        'entrance': (94, 5),
        'ferris_wheel': (100, 40),
        'ferris_junction': (94, 30),
        'pirate_ship': (51, 30),
        'pirate_junction': (56, 35),
        'drop_junction': (56, 60),
        'exit': (124, 25)
    }
```

Figure 26

```python
def spawn_patron(self):
    """Spawn patron"""
    self.total_spawned += 1
    name = f"Patron_{self.total_spawned}"

    # Spawn at entrance
    #x, y = self.waypoints['entrance']
    new_patron = Patron(name, 'o', self.waypoints['entrance'], self)
    self.patrons_list.append(new_patron)
    print(f"{name} entered the park.")
    return new_patron
```

Figure 27

```python
def make_queue_ship(self, patron):
    """
    Make queue for pirate ship (add patron to queue)
    """

    if patron not in self.queue_ship[:]:
        self.queue_ship.append(patron)
        patron.state = "queuing"
        patron.queue_position = len(self.queue_ship) - 1 # gives index of patron in line
        print(f"{patron.name} joined pirate ship at position{len(self.queue_ship)}")


        # horizontal queue

        queue_x = self.waypoints['pirate_ship'][0] + (patron.queue_position * 3) # 3 units apart
        queue_y = self.waypoints['pirate_ship'][1]
        patron.x = queue_x
        patron.y = queue_y
        return (patron.x, patron.y)
```

Figure 28

```python
def make_queue_ferris(self, patron):
    """
    Make queue for ferris wheel (add patron to queue)
    """
    if patron not in self.queue_ferris[:]:
        self.queue_ferris.append(patron)
        patron.state = "queuing"
        patron.queue_position = len(self.queue_ferris) - 1 # gives index of patron in line
        print(f"{patron.name} joined ferris wheel at position{len(self.queue_ferris)}")

        # vertical queue
        queue_x = self.waypoints['ferris_wheel'][0]
        queue_y = self.waypoints['ferris_wheel'][1] - (patron.queue_position * 3) # 3 units apart
        patron.x = queue_x
        patron.y = queue_y
        return (patron.x, patron.y)
```

Figure 29

```python
def make_queue_drop(self, patron):
    """
    Make queue for drop tower (add patron to queue)
    """
    if patron not in self.queue_drop[:]:
        self.queue_drop.append(patron)
        patron.state = "queuing"
        patron.queue_position = len(self.queue_drop) - 1 # gives index of patron in line
        print(f"{patron.name} joined drop tower at position{len(self.queue_drop)}")

        # vertical queue
        queue_x = self.waypoints['drop_junction'][0]
        queue_y = self.waypoints['drop_junction'][1] - (patron.queue_position * 3) # 3 units apart
        patron.x = queue_x
        patron.y = queue_y
        return (patron.x, patron.y)
```

Figure 30

Figure 31

```python
def move_all_patrons(self):
    """
    Update movement of all patrons in list for one timestep
    """
    for patron in self.patrons_list[:]:
        # change patron position
        patron.step_change(self.terrain)

        if patron.state == "leaving":
            distance_from_exit = ((patron.x - self.waypoints['exit'][0])**2 + (patron.y - self.waypoints['exit'][1])**2)**0.5

            if distance_from_exit < 3:
                self.despawn_patron(patron)
```

Figure 32

```
def update_all_rides(self):
    """
    Update both rides
    """
    self.ship_ride.update_ship(self)
    self.ferris_ride.update_ferris(self)
```

Figure 33

```
def despawn_patron(self, patron):
        """
        Despawn patron
        """
        if patron in self.patrons_list:
            self.patrons_list.remove(patron)
            print(f"{patron.name} exited the park.")
```

Figure 34

```
def plot_all_patrons(self, p):
    """
    Plot all active patrons
    """
    for patron in self.patrons_list:
        patron.plot_patron(p)
```

Figure 35

Discussion

The theme park simulation was made in order to show how patrons move around the park to either queue for rides or roam and to demonstrate the varying movements of each ride. The program has two files which provide the simulation when they work together.

ThemePark.py contains the classes Ride, Terrain, Patron and Park_Operations.

In class Ride(), there is plot_ship(self, plt, pos) , drop_tower(self, p) and plot_ferrisWheel(self, p, pos)
methods directly plot the rides. But in the case of plot_ship(self, plt, pos) and plot_ferrisWheel(self, p, pos), they plot the Pirate Ship and Ferris wheel before scaling and translating them according to the width and height of the ride barrier. As for the Drop tower, it is directly coded into the ride barrier. As for the movements of the rides, the swinging motion of the Pirate Ship is due to the use of a rotation angle which is converted into radian before being used in the formula. The Ferris wheel uses a rotation formula with sine and cosine with the

angle in radian. But the Drop tower has an upwards and downwards movement thus adding more variety to the ride movements.

The class Terrain() oversees the terrain of 'Adventure World' theme park, I.e., it displays the sky which is in a Graphics Interchange Format (GIF). Moreover, it has load_gif_frames(self, filename) and update_frame(self, timestep) which load and update its frames to ensure it is properly displayed. Also, load_gif_frames(self, filename) makes use of Error Handling to anticipate the file not loading or if it is not found. Furthermore, a grid system making use of numpy.zeros was used to provide colours to the terrain such as green for grass, grey for the background of the rides, yellow for the paths and brown for the fence around the park.

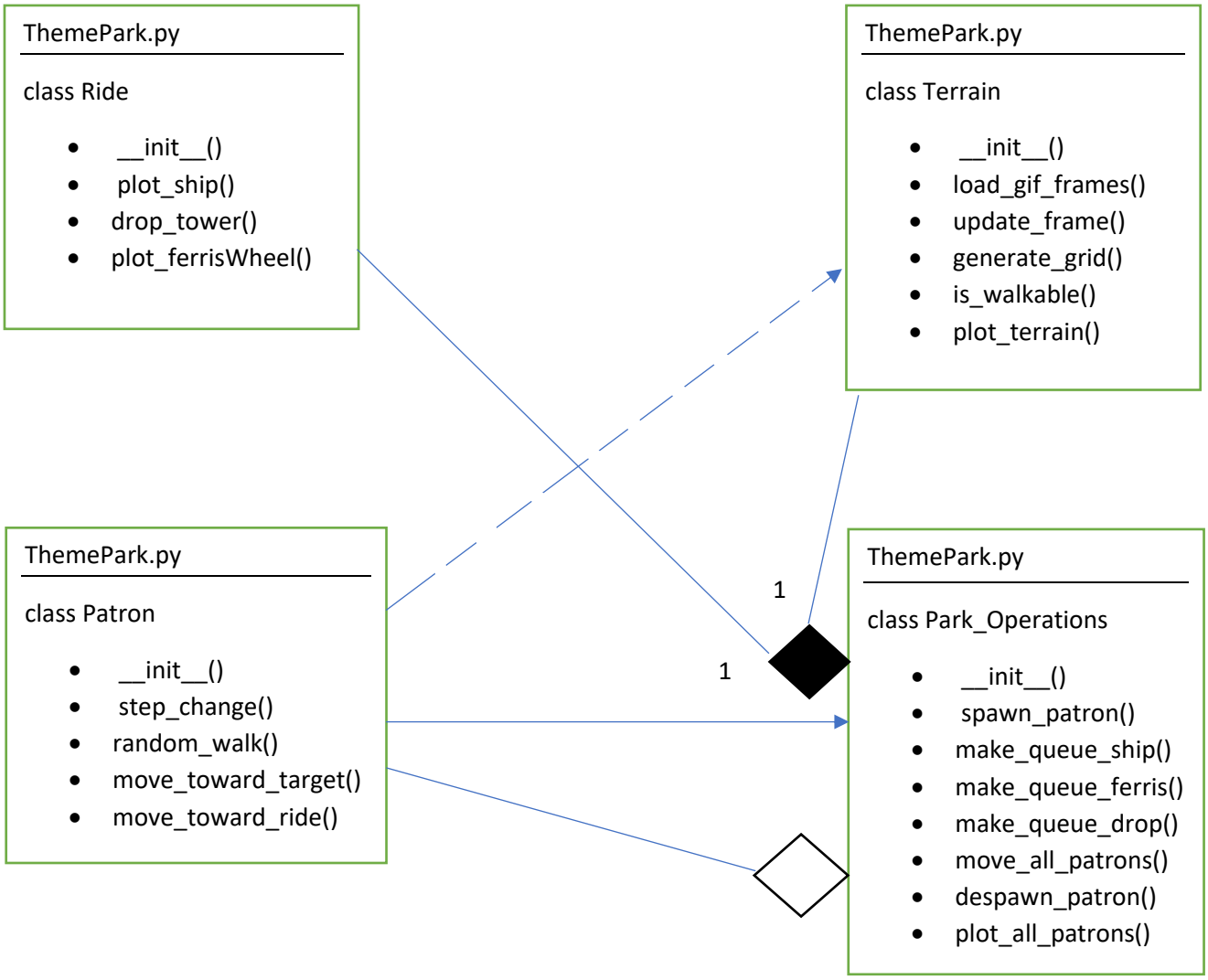
The class Patron() mainly contains methods which allow patrons to move depending on their states. Adding to that, the step_change(self, terrain) method allows patrons to detect rides if they wish to ride attractions but it also helps to prevent collision when the patrons are roaming.

The class Park_Operations() manages all patron and ride operations in the park. It allows patrons to spawn at the entrance using spawn_patron(self), queue for their respective rides using methods such as make_queue_ship(self, patron), make_queue_ferris(self, patron) and make_queue_drop(self, patron). The move_all_patrons(self) makes each patron appended to a list at the start of the simulation move in their respective directions according to their state.

Matplotlib is used to simulate the theme park.

UML Diagram

The UML diagram below illustrates the relationships between the main classes – Ride, Terrain, Patron and Park_Operations.



Module Description

ThemePark.py

Module Name	Module Description

class Ride()	This is the class which contains the methods for the rides. It has a method for each ride which allows to plot and move them.
__init__(self, name, size, position, duration, park, capacity=8, animation_count=0)	This method initialises the environment while referring to parameters.
plot_ship(self, plt, pos)	It plots the boundary, the pirate ship and the swinging motion.
drop_tower(self, p)	It plots the boundary, the drop tower and the up and down motion.
plot_ferrisWheel(self, p, pos)	It plots the boundary, the ferris wheel and the rotation.

ThemePark.py

Module Name	Module Description
class Terrain()	This class contains methods to generate the terrain of the theme park.
__init(self, boundary, size, entrance, exit, gif_file=None)	This method initialises the environment while referring to parameters.
load_gif_frames(self, filename)	It loads all frames from gif file.
update_frame(self, timestep)	It shows which GIF frame should be displayed.
generate_grid(self)	It generate terrain grid with numbers representing different areas. The different areas are given a colour.
is_walkable(self, x, y)	It verifies if position (x, y) is walkable ().
plot_terrain(self, p)	It stores pictures and gif and display them on terrain.

ThemePark.py

Module Name	Module Description
class Patron()	This class contains all motions related to how patrons will move.
__init__(self, name, marker, position, park)	This method initialises the environment while referring to parameters.
step_change(self, terrain)	It updates patron position and verifies terrain for walkability using direct coordinate access.
random_walk(self, terrain)	It moves patron randomly (roaming) while stay on walkable terrain.
move_toward_target(self, target, terrain)	It moves patron 1 step toward target without going into ride boundaries. It is used for roaming patrons.

move_toward_ride(self, target)	It moves patron 1 step toward target and can possibly go through ride boundaries. It is used for queuing patrons.

ThemePark.py

Module Name	Module Description
class Park_Operations()	This class manages all patron and ride operations in park.
__init__(self, terrain)	This method initialises the environment while referring to parameters.
spawn_patron(self)	It spawns patron and adds them to a list.
make_queue_ship(self, patron)	It makes a queue for pirate ship.
make_queue_ferris(self, patron)	It makes a queue for ferris wheel.
make_queue_drop(self, patron)	It makes a queue for drop tower.
move_all_patrons(self)	It updates movement of all patrons in list for one timestep.
despawn_patron(self, patron)	It despawns patron and removes them from a list containing active patrons.
plot_all_patrons(self, p)	It plots all active patrons from the list.

Showcase

The theme park simulation program illustrates the choice of patrons' of either roaming or queueing through a prompt given in the terminal after entering the number of patrons which is also entered through a prompt. The third prompt appears only if the user chooses to make the patron go riding. It gives three different rides as options. Only after entering the input for each prompt, that the simulation starts using matplotlib. The roaming patrons walk randomly while the patrons which have the state of riding go into their queuing positions allocated to them.

Scenario 1

This scenario was chosen to show queueing and roaming interactions.

The inputs are in the figure below:

```
Enter the number of patrons: 8
Patron_1 entered the park.
What do you want to do ?: a) roaming b) riding: a
Patron_2 entered the park.
What do you want to do ?: a) roaming b) riding: a
Patron_3 entered the park.
What do you want to do ?: a) roaming b) riding: b
Which ride do you want ? - (a)- Pirate Ship (b)- Ferris Wheel (c)- Drop Tower: b
Patron_4 entered the park.
What do you want to do ?: a) roaming b) riding: b
Which ride do you want ? - (a)- Pirate Ship (b)- Ferris Wheel (c)- Drop Tower: b
Patron_5 entered the park.
What do you want to do ?: a) roaming b) riding: b
Which ride do you want ? - (a)- Pirate Ship (b)- Ferris Wheel (c)- Drop Tower: a
Patron_6 entered the park.
What do you want to do ?: a) roaming b) riding: b
Which ride do you want ? - (a)- Pirate Ship (b)- Ferris Wheel (c)- Drop Tower: a
Patron_7 entered the park.
What do you want to do ?: a) roaming b) riding: b
Which ride do you want ? - (a)- Pirate Ship (b)- Ferris Wheel (c)- Drop Tower: c
Patron_8 entered the park.
What do you want to do ?: a) roaming b) riding: b
Which ride do you want ? - (a)- Pirate Ship (b)- Ferris Wheel (c)- Drop Tower: c
```

Figure 36

- In scenario 1, the number of patrons who enter the theme park is eight. (Refer to figure 27)
- Two of them will go directly queue to the Pirate ship horizontally while two others go to the Ferris wheel to be queued vertically. (Refer to figure 28) (Refer to figure 29)
- Another two will queue vertically outside of the boundary of Drop tower. (Refer to figure 30)
- The remaining two will roam in the park. (Refer to figure 31)

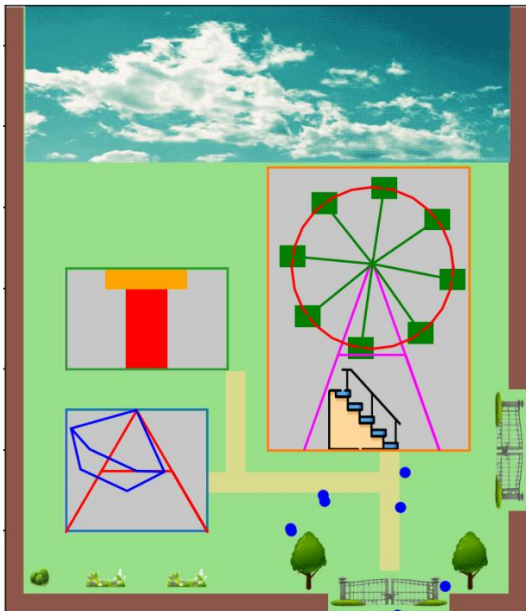


Figure 37

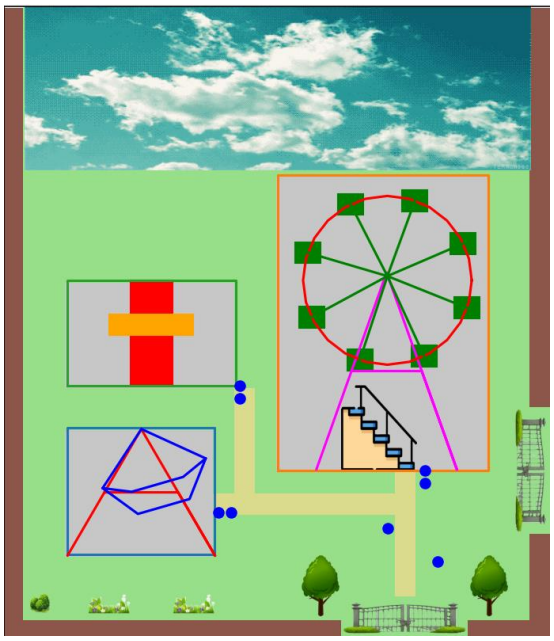


Figure 38

As for the result, the roaming patrons avoid ride areas and fences due to the grid in the class Terrain().

Scenario 2

This scenario was chosen to observe how roaming patrons behave in the theme park.

The inputs are given in the figure below:

```
Enter the number of patrons: 5
Patron_1 entered the park.
What do you want to do ?: a) roaming b) riding: a
Patron_2 entered the park.
What do you want to do ?: a) roaming b) riding: a
Patron_3 entered the park.
What do you want to do ?: a) roaming b) riding: a
Patron_4 entered the park.
What do you want to do ?: a) roaming b) riding: a
Patron_5 entered the park.
What do you want to do ?: a) roaming b) riding: a
```

Figure 39

- In scenario 1, the number of patrons who enter the theme park is five. (Refer to figure 27)
- They roam for a total of 300 timesteps. (Refer to figure 22) (Refer to figure 23) (Refer to figure 31)
- They avoid collision with fences and ride areas, and they also avoid the trees and the sky. (Refer to figure 17) (Refer to figure 18)

- But as one of the patrons get closer to the exit, they are despawned, i.e., they exit the theme park. (Refer to figure 22) (Refer to figure 34)

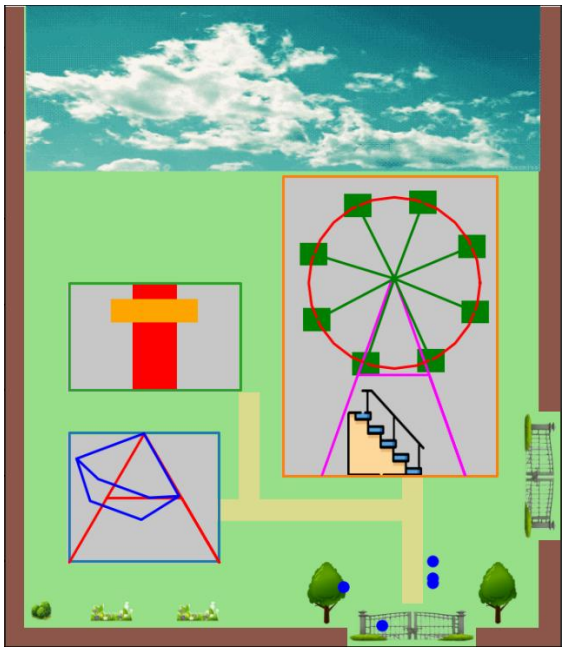


Figure 40

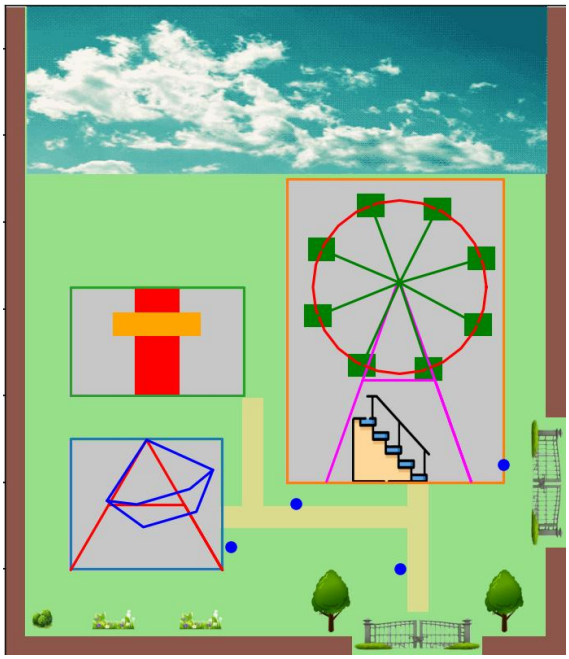


Figure 41

As a result, it shows that roaming patrons can safely explore the park and exit through the designated gate if they are within the required distance which in this case is less than five units.

Scenario 3

This scenario was chosen for the observation of the behaviour of patrons if they are to get in line for each available attraction in the theme park.

The input for this scenario is listed below:

```
Enter the number of patrons: 15
Patron_1 entered the park.
What do you want to do ?: a) roaming b) riding: b
Which ride do you want ? - (a)- Pirate Ship (b)- Ferris Wheel (c)- Drop Tower: b
Patron_2 entered the park.
What do you want to do ?: a) roaming b) riding: b
Which ride do you want ? - (a)- Pirate Ship (b)- Ferris Wheel (c)- Drop Tower: b
Patron_3 entered the park.
What do you want to do ?: a) roaming b) riding: b
Which ride do you want ? - (a)- Pirate Ship (b)- Ferris Wheel (c)- Drop Tower: b
Patron_4 entered the park.
What do you want to do ?: a) roaming b) riding: b
Which ride do you want ? - (a)- Pirate Ship (b)- Ferris Wheel (c)- Drop Tower: b
Patron_5 entered the park.
What do you want to do ?: a) roaming b) riding: b
Which ride do you want ? - (a)- Pirate Ship (b)- Ferris Wheel (c)- Drop Tower: b
Patron_6 entered the park.
What do you want to do ?: a) roaming b) riding: b
Which ride do you want ? - (a)- Pirate Ship (b)- Ferris Wheel (c)- Drop Tower: a
Patron_7 entered the park.
What do you want to do ?: a) roaming b) riding: b
Which ride do you want ? - (a)- Pirate Ship (b)- Ferris Wheel (c)- Drop Tower: a
Patron_8 entered the park.
What do you want to do ?: a) roaming b) riding: b
Which ride do you want ? - (a)- Pirate Ship (b)- Ferris Wheel (c)- Drop Tower: a
Patron_9 entered the park.
What do you want to do ?: a) roaming b) riding: b
Which ride do you want ? - (a)- Pirate Ship (b)- Ferris Wheel (c)- Drop Tower: a
Patron_10 entered the park.
What do you want to do ?: a) roaming b) riding: b
Which ride do you want ? - (a)- Pirate Ship (b)- Ferris Wheel (c)- Drop Tower: a
Patron_11 entered the park.
What do you want to do ?: a) roaming b) riding: b
Which ride do you want ? - (a)- Pirate Ship (b)- Ferris Wheel (c)- Drop Tower: c
Patron_12 entered the park.
What do you want to do ?: a) roaming b) riding: b
Which ride do you want ? - (a)- Pirate Ship (b)- Ferris Wheel (c)- Drop Tower: c
Patron_13 entered the park.
What do you want to do ?: a) roaming b) riding: b
Which ride do you want ? - (a)- Pirate Ship (b)- Ferris Wheel (c)- Drop Tower: c
Patron_14 entered the park.
What do you want to do ?: a) roaming b) riding: b
Which ride do you want ? - (a)- Pirate Ship (b)- Ferris Wheel (c)- Drop Tower: c
```

Figure 42

```
Patron_15 entered the park.
What do you want to do ?: a) roaming b) riding: b
Which ride do you want ? - (a)- Pirate Ship (b)- Ferris Wheel (c)- Drop Tower: c
```

Figure 43

- In scenario 3, the number of patrons which enter the park is fifteen. (Refer to figure 27)
- The patrons move towards their respective ride and queue in an orderly manner near the rides. (Refer to figure 35)
- Five patrons queue horizontally for the Pirate ship. (Refer to figure28)
- Five patrons queue vertically for the Ferris wheel. (Refer to figure 29)
- Five patrons queue vertically for the Drop tower. (Refer to figure 30)
- During simulation, every single patron stays in their respective queues demonstrating good queue behaviour and correct queue positioning.
- The three rides continue to move without any problem while the patrons wait in the queue. (Refer to figure 11) (Refer to figure 12) (Refer to figure 13)

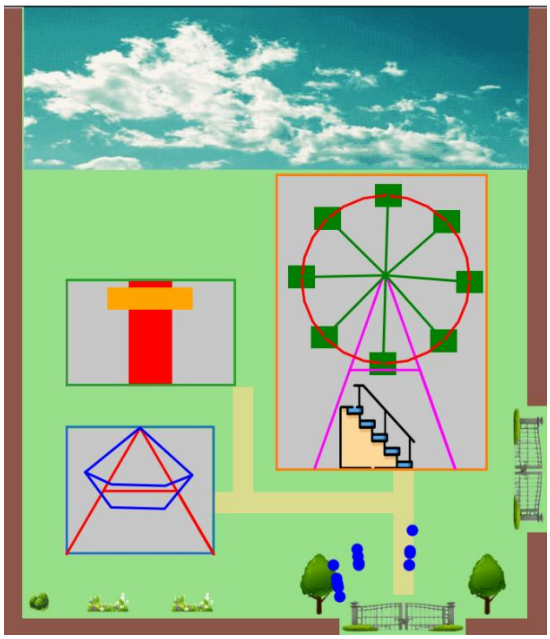


Figure 44

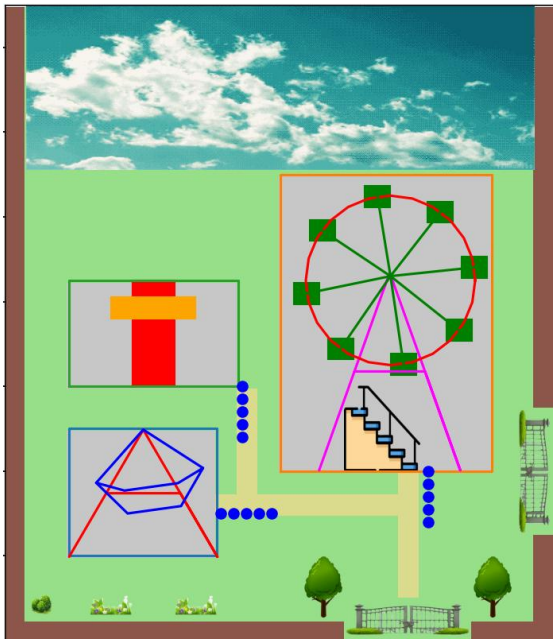


Figure 45

As a result, the queuing system is shown to work correctly for all attractions, and it is also shown that the rides moving at the same time does not affect the queue positions.

Conclusion

A theme park simulation program was created with different rides which have varying movement patterns in this assignment. Object-oriented programming was used to create the theme park simulation. The animation demonstrates two different states of patrons which are roaming and queuing. Additionally, three different rides with three different movements were

simulated along with daytime using GIF. The theme park successfully shows how the patrons interact with the rides and obstacles.

The features listed down below were implemented with success:

- File handling
- Different types of rides with different structure and movement
- Patron queuing system
- Avoiding obstacles
- Time of day simulation
- Error handling

Future Work

In the future, I would like to upgrade the program in the following ways:

- Use a CSV file to generate terrain
- Add more rides with more complex movements
- Replace the patrons which are in dots with stickmen
- Add sound effects
- Make the patrons get on the rides
- Add more decorations
- Add moving pets such as dogs and cats

The above improvements would render the simulation more enjoyable and interesting.

References

Cleanpng.Com. n.d. 'Lush Green Bushes in Nature Scene - Unlimited Download. Cleanpng.Com.' Accessed 12 October 2025. https://www.cleanpng.com//png-bushes-bushes-green-leaves-grass-field-group-of-bu-7996057/.

COMP1005. "Fundamentals of Programming: Practical Test 3." Accessed September 18th, 2025 via Blackboard

'Deciduous Tree on Emojidex 1.0.34'. n.d. Accessed 12 October 2025. https://emojipedia.org/emojidex/1.0.34/deciduous-tree.

'Gate Emoji | AI Emoji Generator'. n.d. Accessed 12 October 2025. https://www.emojis.com/emoji/gate-gA4BS2JXSx.

GeeksforGeeks. n.d. Accessed 18 September 2025. 'NumPy Tutorial'. https://www.geeksforgeeks.org/python/numpy-tutorial/.

GeeksforGeeks. n.d. Accessed 18 September 2025. 'Python Random Module'. https://www.geeksforgeeks.org/python/python-random-module/.

'Matplotlib.Pyplot — Matplotlib 3.5.3 Documentation'. n.d. Accessed 18 September 2025. https://matplotlib.org/3.5.3/api/_as_gen/matplotlib.pyplot.html.

'Matplotlib.Colors.ListedColormap — Matplotlib 3.10.7 Documentation'. n.d. Accessed 19 September 2025. https://matplotlib.org/stable/api/_as_gen/matplotlib.colors.ListedColormap.html.

NicePNG.Com. n.d. 'Free Png Download Grass And Flowers Png Images Background - Flower Field Clipart Png Transparent PNG - 850x444 - Free Download on NicePNG'. Accessed 12 October 2025. https://www.nicepng.com/ourpic/u2y3q8a9r5q8e6a9_free-png-download-grass-and-flowers-png-images/.

Partly Cloudy Sky GIF - Find & Share on GIPHY. n.d. Accessed 12 October 2025. https://giphy.com/gifs/sky-clouds-cloud-G1T5M0qT6ZJlu.

Pillow (PIL Fork). n.d. 'Image Module'. Accessed 18 September 2025. https://pillow.readthedocs.io/en/stable/reference/Image.html.

Vecteezy. n.d. 'Download A Stairway with a Handrail and a Handrail for Free'. Accessed 12 October 2025. https://www.vecteezy.com/vector-art/54727238-a-stairway-with-a-handrail-and-a-handrail.